



TASK

Event Handling

Visit our website

Introduction

In JavaScript, event handling allows web pages to respond to user interactions and browser actions. Events, such as clicks, keyboard presses, and page load completions, trigger specific functions in the code. This process enables dynamic and interactive web experiences, where the page reacts to actions like submitting a form, clicking a button, or hovering over an element. Event handling is a core feature of modern web development. Understanding how events work and how to manage them is crucial for building interactive and engaging websites.

EVENTS

An event is an action that triggers a response from your program. You may have come across DOM (Document Object Model) events, which are actions initiated by the user, like clicking a button or typing in a textbox, as well as actions triggered by the browser, such as when a web page has fully loaded.

The most common events you will deal with when getting started are:

Event	Description
onchange	Some HTML elements have been modified.
onclick	An HTML element has been clicked on.
onmouseover	An HTML element was hovered over.
onmouseout	Mouse cursor moves off HTML element.
onkeydown	A keyboard button is pressed.
onload	The HTML page has finished loading.

Other events are listed and explained [here](#).

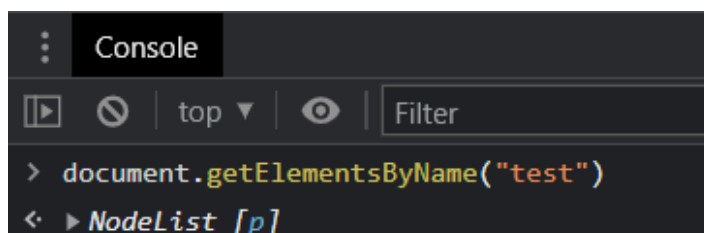
Every DOM element object in JavaScript has the DOM event methods available to them. We can see this by doing the following. Copy the following code into an HTML file and open it in the browser:

```
<!DOCTYPE html>
<html>
  <body>
    <p name="test">This is a test</p>
  </body>
</html>
```

Open the developer tools in your browser and enter the following command in the console:

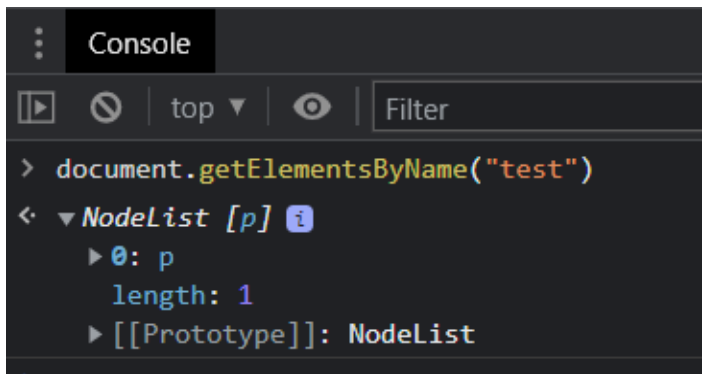
```
document.getElementsByName("test");
```

The above DOM method will retrieve all element objects that have the name “test” and put them in an array. You will get the following output after running this DOM method:

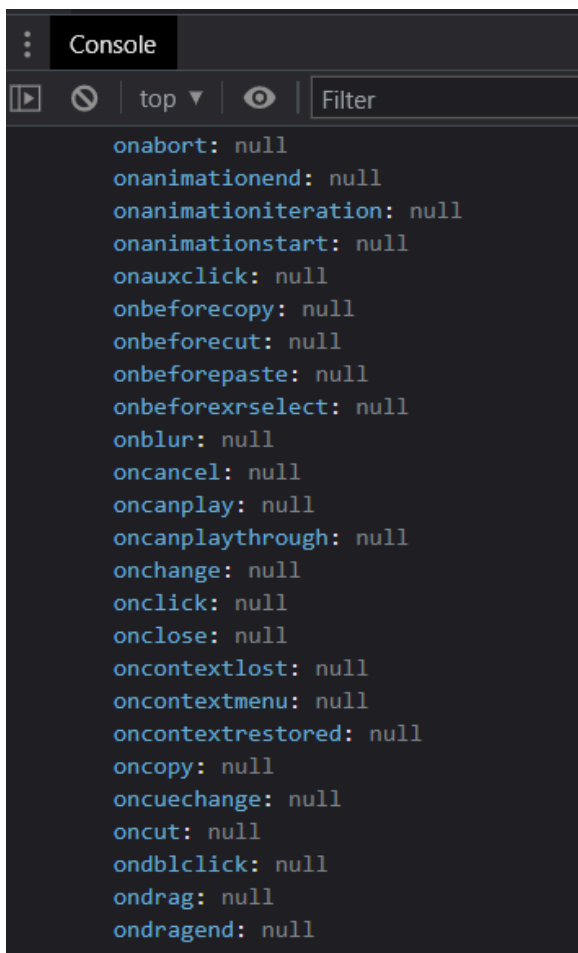


You will see that we have a NodeList array with the single paragraph object that we named “test”.

When we expand the NodeList by clicking on the arrow, we get the following:



If we click on the arrow next to the `0` index element (our paragraph object, `p`), we can see all of the attributes and methods attached to the paragraph object. It is very important to understand that every element in the DOM is an object. Here is a snippet of the available methods:



You will notice that all of the event methods have the value **null**. This is because there are no functions attached to them as yet. Let's attach a function to one of these event methods. We can create a JS file named "test.js" with the following code:

```
// Grabbing the object at the 0 index of the array and setting it to a variable.  
let selectedElement = document.getElementsByName('test')[0];  
  
// Assigning an anonymous function to the onclick method as a callback function  
selectedElement.onclick = function() {  
    console.log("The paragraph was clicked");  
};
```

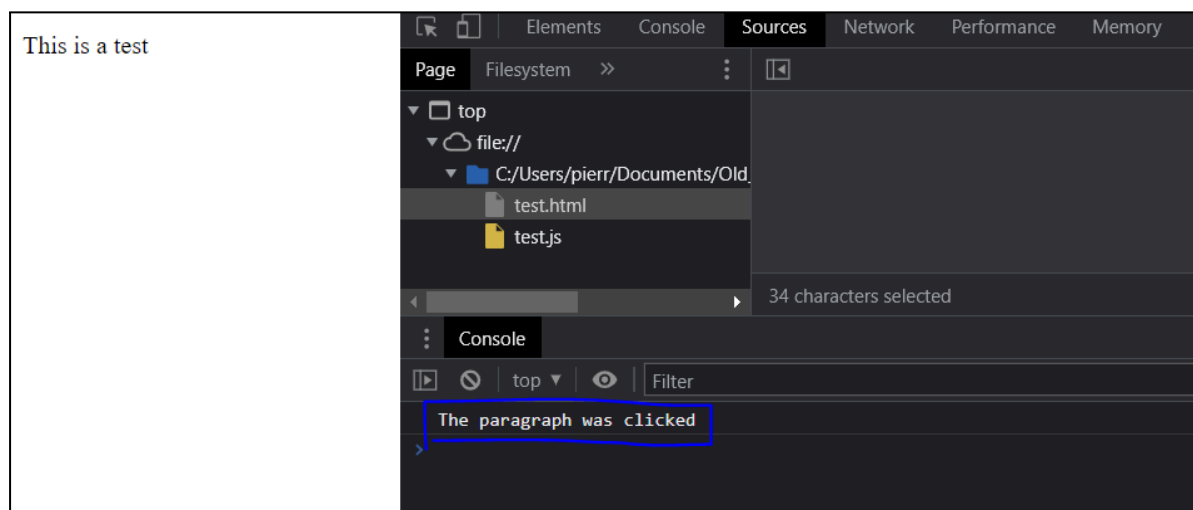
We can then change the HTML file to the following and import the external JavaScript:

```
<!DOCTYPE html>  
<html>  
  <body>  
    <p name="test">This is a test</p>  
    <script type="text/javascript" src="test.js"></script>  
  </body>  
</html>
```

If we open the HTML file now and look again at the paragraph object in the console, we will see that a function has now been assigned to the **onclick** method as a callback function. When the method is called, the callback function is executed.

```
onchange: null
▼ onclick: f ()
  arguments: null
  caller: null
  length: 0
  name: ""
  ► prototype: {constructor: f}
    [[FunctionLocation]]: test.js:5
  ► [[Prototype]]: f ()
  ► [[Scopes]]: Scopes[1]
onclose: null
oncontextlost: null
oncontextmenu: null
oncontextrestored: null
oncopy: null
oncuechange: null
```

If we click on the paragraph in the browser, we will get the following output in the console:

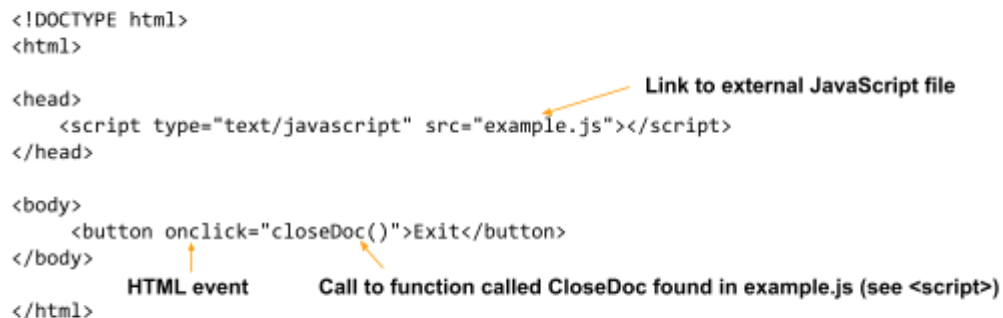


We can write JavaScript code that tells your browser what you want it to do when it realises that a certain event has occurred. This code could be written in a JavaScript function.

JAVASCRIPT FUNCTIONS AS EVENT HANDLERS

JavaScript is a key tool for responding to user interactions and other events on a website. To use it, you simply identify the DOM event and associate it with a JavaScript function that should be executed when the event occurs. For example:

```
<!DOCTYPE html>
<html>
  <head>
    <script type="text/javascript" src = "example.js"></script>
  </head>
  <body>
    <button onclick="closeDoc()">Exit</button>
  </body>
</html>
```



The diagram shows the same HTML code as above, but with three orange arrows pointing to specific parts and labels:

- An arrow points to the `src="example.js"` attribute in the `<script>` tag, with the label "Link to external JavaScript file".
- An arrow points to the `onclick="closeDoc()"` attribute in the `<button>` tag, with the label "HTML event".
- An arrow points to the `closeDoc()` function call in the `onclick` attribute, with the label "Call to function called CloseDoc found in example.js (see <script>)"

The above is an example of an HTML file where we want our browser to listen for a certain event. Notice that in the `<head>` element, there is a `<script>` element that refers to the external JavaScript file called **example.js**. Also, notice the button element: here we instruct the browser to call the function named `closeDoc()` when the button is clicked. The `closeDoc()` function is declared in the external JavaScript file, **example.js**, as shown above.

The JavaScript file called **example.js** would likely contain the following function that will be called and executed when the button mentioned in the HTML page is clicked:

```
function closeDoc(){
    alert("You are closing this page!");
    window.close();
}
```

Copy the above HTML code into a file named "test.html" and the JavaScript code into a file named example.js in the same folder. Open the HTML file in the browser, click on the exit button and observe what happens. You will first see the alert. Once the alert is closed, the **window.close()** function is called. This is a built-in JavaScript function that closes the current tab that you are working in.

We could also add a function to an event in an element object in the following manner using DOM manipulation and event listeners.

Create an HTML file named **example.html** and add the following code to the file. We are creating two elements. A button and a paragraph element. When the button element is clicked, we want the paragraph to display some text:

```
<!DOCTYPE html>
<html>
  <body>
    <button id="showMoreButton">Show More</button>
    <p id="info"></p>
    <script type="text/javascript" src="example.js"></script>
  </body>
</html>
```


We can then put the following code in a file named “example.js” and this is where the magic happens:

```
// Selecting the button and paragraph elements by their 'id' and assigning those objects to variables
let showMoreButton = document.getElementById('showMoreButton');
let information = document.getElementById('info');

// Append text to the paragraph element when called
function showMoreDetails() {
  information.innerHTML += "some more information";
}

/* Using addEventListener to assign the function to the button, which triggers on a click event. Unlike onclick, we use just 'click' without the 'on' prefix. */
showMoreButton.addEventListener('click', showMoreDetails);
```

THE EVENT OBJECT

JavaScript represents events as event objects, which store information about the event and its context. When an event is triggered, the event object is passed to the handler function. This object contains useful properties, such as identifying which element triggered the event. It can also be used to create more complex behaviours, like distinguishing whether a button was clicked while holding down a key, which can be helpful for tasks like debugging or managing interactions.

It is important to note that the event object is an optional argument and that the event handler function does not need it to trigger logic. In codebases, you may see the event object argument named as one of the following:

- “e”
- “evt”
- or “event”

These are all just synonyms for the same thing, but the usual convention that is most commonly found is simply the letter e. Let's take a look at a code example:

```
<html>
  <head>
    <title>My Webpage</title>
  </head>

  <body>
    <button id="genericButton">Click This</button>
    <script>
      // Retrieving the button element
      let button = document.querySelector("#genericButton");
      // Passing event object "e" as argument to event handler function
      button.addEventListener("click", function (e) {
        console.log(e); // Outputting the event object to the console
      })
    </script>
  </body>
</html>
```

In the code snippet above, we are passing an event object as an argument to an anonymous function being used to trigger logic when the button is clicked. If you paste this code into an html file, open in the browser, click the button, and open the developer tools, you will find the following output in the console (which you can see on the following page):

```
Console
Filter
▼ PointerEvent {isTrusted: true, pointerId: 1, width: 1, height: 1, pressure: 0, ...} ⓘ
  isTrusted: true
  altKey: false
  altitudeAngle: 1.5707963267948966
  azimuthAngle: 0
  bubbles: true
  button: 0
  buttons: 0
  cancelBubble: false
  cancelable: true
  clientX: 58
  clientY: 19
  composed: true
  ctrlKey: false
  currentTarget: null
  defaultPrevented: false
  detail: 1
  eventPhase: 0
  fromElement: null
  height: 1
  isPrimary: false
  layerX: 58
  layerY: 19
  metaKey: false
  movementX: 0
  movementY: 0
  offsetX: 48
  offsetY: 9
  pageX: 58
  pageY: 19
  ▶ path: (5) [button#genericButton, body, html, document, Window]
  pointerId: 1
  pointerType: "mouse"
  pressure: 0
  relatedTarget: null
  returnValue: true
  screenX: 58
  screenY: 122
  shiftKey: false
  ▶ sourceCapabilities: InputDeviceCapabilities {firesTouchEvents: false}
  ▶ srcElement: button#genericButton
  tangentialPressure: 0
  ▶ target: button#genericButton
  tiltX: 0
  tiltY: 0
  timeStamp: 1827.2000000029802
  toElement: null
  twist: 0
  type: "click"
  ▶ view: Window {window: Window, self: Window, document: document, name: '', location: Location, ...}
  which: 1
```

The event object contains various properties that provide details about the event, such as its location on the screen, the event type, the timestamp, and whether modifier keys like **Ctrl** or **Shift** were pressed during the event. You can find an explanation of common properties in the event object and what they mean in this helpful [resource](#).

Let's take a quick look at an example of what can be done with the event object:

```
<html>
  <head>
    <title>My Web Page</title>
  </head>
  <body>
    <button id="genericButton">Click This</button>
    <script>
      // Retrieving the button element
      let button = document.querySelector("#genericButton");
      // Passing event object "e" as argument to event handler function
      button.addEventListener("click", function (e) {
        /* Check whether the shift key was pressed or not shiftKey property
        would be true if it was */
        if (e.shiftKey) {
          console.log("The shift key was pressed");
        } else {
          console.log("The shift key was not pressed");
        }
      });
    </script>
  </body>
</html>
```

Copy and paste the above code in an HTML file, open it in the browser and check the console for what happens if a **Shift** key is pressed while clicking on the button or not. You can imagine the possibilities here. For an advanced understanding of events in JavaScript, check out at this [resource](#) on event bubbling and capturing.

THE EVENT LOOP

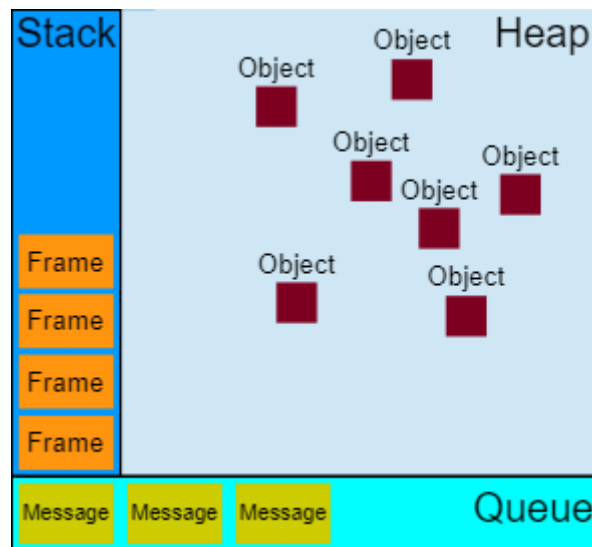
JavaScript is single-threaded, meaning it can only execute one task at a time. However, it uses an event loop to handle multiple tasks efficiently, enabling asynchronous operations like responding to user interactions, or fetching data from an API. The event loop works by:

Executing code in the call stack: The call stack is where JavaScript keeps track of the functions that are currently being executed. When a function is called, it is pushed onto the call stack. Once the function finishes executing, it is removed from the stack, and the next function in line is executed.

Delegating tasks to the callback queue: Some tasks, like event listeners (e.g., button clicks) are asynchronous, meaning they do not need to be executed immediately. These tasks are handled by the browser and placed into the callback queue when they are ready to run. The callback queue holds tasks that are waiting to be executed but cannot run until the call stack is empty.

Checking the call stack: The event loop continuously checks if the call stack is empty. If there are no functions currently executing, it takes the next task from the callback queue and pushes it onto the call stack for execution.

The heap: The heap stores dynamically allocated memory, including objects and data, that tasks in the call stack may use during execution. It supports the event loop by providing the necessary resources for these tasks.



The Event Loop (MDN Web Docs, n.d.)

By using this system, JavaScript can handle multiple operations without blocking the main thread. This enables asynchronous tasks like waiting for user interaction while ensuring the application remains responsive.

Instructions

Open the accompanying example files in Visual Studio Code and read through the comments before attempting these tasks.



Practical task

In this task, you will be required to modify your files from the previous task. Follow these steps:

- **JavaScript:**

- Modify the `displayItems` function:
 - Each created list element should contain a `<span>` element with a class named `close`.
 - Each `span` element should have a value of `'\u00D7'`, which represents the character 'x' in unicode.
 - As the last line of this function, call the `delete` function (described below) to delete an item from the shopping list.
- Create a function to Add Items:
 - Create a function which will update the `grocery items` array by getting the value of the text in the `<input>` tag and adding it to the array.
 - If the input text field is empty, display an alert to the user indicating that they should insert an item. Else, add the input text to the array.
 - Once the item has been added to the array or the alert displayed, reset the input text's value to an empty string.
 - As the last line of this function, call the `displayItems` function to display the updated array items.
- Create a function to Delete Items:
 - Add a click event listener to each `<span>` element with a `close` class.
 - When the event is triggered, delete the item from the array.
- Toggle items:
 - Add a click event listener to the html element with the ID of `itemList`.
 - If the event's tag name is a list tag, toggle a `checked` class on the event element.
- Keyboard functionality:

- Add a **key up** event listener to the HTML element with an **input** ID. Use the event key 'Enter' to trigger a **click()** function on the element with an **addButton** ID.
- **CSS:**
 - All elements with a **checked** class should have styling which indicates that the item has been checked off the shopping list.
- **HTML:**
 - Add an **onclick** attribute to the HTML element with an **addButton** ID. When this element is clicked, the function which updates the shopping list should be called.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
